

CSE114 – Labwork 9.1

Labwork 9.1 – Recursive Operations

General Information and Submission Rules

By the end of the labwork, you must submit it to Yulearn as:

```
studentID_fullname_labwork9.c
```

Important:

- Incorrect naming or file format will result in a loss of 5 points from your total grade.
- Only use concepts covered in the labs so far.
- **All functions in this labwork must be implemented using recursion (no loops allowed in recursive functions).**

Question 1 – Display Message (10p) EASY

Create a new function `welcome` that displays a welcome message and a list of all available operations to the user.

This function will expect a decision from a user so that it will execute related functions based on this choice until the user exits.

Make sure to print each line separately.

```
Welcome to Labwork 9 Recursive Operations! By using this machine you can
do:
1. Calculate Factorial
2. Sum of Digits
3. Power Calculation
4. Print Numbers (Ascending)
5. Print Numbers (Descending)
6. Multiplication (Recursive)
7. Division (Recursive)
8. Count Digits
9. Reverse Number
10. Exit
Make your choice:
```

Question 2 – Calculate Factorial (10p) EASY

Create a recursive function `factorial` that takes:

- an integer n

The function should calculate and return $n!$ (n factorial) using recursion.

Mathematical Definition:

```
n! = n × (n-1)!  
0! = 1  
1! = 1
```

Additionally, print each recursive step to show the calculation process.

Example:

```
>> Enter a number: 5  
Calculating: 5! = 5 × 4!  
Calculating: 4! = 4 × 3!  
Calculating: 3! = 3 × 2!  
Calculating: 2! = 2 × 1!  
Calculating: 1! = 1  
Result: 5! = 120
```

Question 3 – Sum of Digits (10p) EASY

Create a recursive function `sum_of_digits` that takes:

- an integer n

The function should calculate the sum of all digits of the given number using recursion.

Hint:

- Use $n \% 10$ to get the last digit
- Use $n / 10$ to remove the last digit

Example:

```
>> Enter a number: 12345  
Sum of digits: 1 + 2 + 3 + 4 + 5 = 15
```

Another Example:

```
>> Enter a number: 9876  
Sum of digits: 9 + 8 + 7 + 6 = 30
```

Question 4 – Power Calculation (10p) EASY

Create a recursive function `power` that takes:

- the base number
- the exponent

The function should calculate `baseexponent` using recursion.

Mathematical Definition:

$$a^b = a \times a^{(b-1)}$$
$$a^0 = 1$$

Example:

```
>> Enter base: 2
>> Enter exponent: 5
Result: 2^5 = 32
```

Question 5 – Print Numbers Ascending (10p) EASY

Create a recursive function `print_ascending` that takes:

- a starting number `start`
- an ending number `end`

The function should print all numbers from `start` to `end` in ascending order using recursion.

Example:

```
>> Enter start: 1
>> Enter end: 5
Numbers: 1 2 3 4 5
```

Question 6 – Print Numbers Descending (10p) EASY

Create a recursive function `print_descending` that takes:

- a starting number `start`
- an ending number `end`

The function should print all numbers from `start` down to `end` in descending order using recursion.

Example:

```
>> Enter start: 10
>> Enter end: 5
Numbers: 10 9 8 7 6 5
```

Question 7 – Recursive Multiplication (15p) MEDIUM

Create a recursive function `multiply` that takes:

- two integers `a` and `b`

The function should calculate `a × b` **without using the `*` operator**. Use only addition and recursion.

Mathematical Definition:

```
a × b = a + a × (b-1)
a × 0 = 0
a × 1 = a
```

Example:

```
>> Enter first number: 4
>> Enter second number: 5
Calculating: 4 × 5 = 4 + (4 × 4)
Calculating: 4 × 4 = 4 + (4 × 3)
Calculating: 4 × 3 = 4 + (4 × 2)
Calculating: 4 × 2 = 4 + (4 × 1)
Calculating: 4 × 1 = 4
Result: 4 × 5 = 20
```

Question 8 – Recursive Division (15p) MEDIUM

Create a recursive function `divide` that takes:

- two integers `a` (dividend) and `b` (divisor)

The function should calculate `a / b` (integer division) **without using the `/` operator**. Use only subtraction and recursion.

Mathematical Definition:

```
a / b = 1 + (a-b) / b    (if a >= b)
a / b = 0                (if a < b)
```

Example:

```
>> Enter dividend: 17
>> Enter divisor: 5
Calculating: 17 / 5 = 1 + (12 / 5)
Calculating: 12 / 5 = 1 + (7 / 5)
Calculating: 7 / 5 = 1 + (2 / 5)
Calculating: 2 / 5 = 0
Result: 17 / 5 = 3
```

Question 9 – Count Digits (15p) MEDIUM

Create a recursive function `count_digits` that takes:

- an integer `n`

The function should count and return the number of digits in `n` using recursion.

Mathematical Definition:

```
count(n) = 1 + count(n/10)    (if n >= 10)
count(n) = 1                  (if n < 10)
```

Example:

```
>> Enter a number: 12345
Number of digits: 5
```

Another Example:

```
>> Enter a number: 1000000
Number of digits: 7
```

Question 10 – Reverse Number (20p) HARD

Create a recursive function `reverse_number` that takes:

- an integer `n`
- a pointer to store the reversed result

The function should reverse the digits of the given number using recursion.

Hint: You may need a helper function or use the `count_digits` function.

Example:

```
>> Enter a number: 12345
Reversed number: 54321
```

Another Example:

```
>> Enter a number: 1000
Reversed number: 1
```

BONUS Question – GCD (Greatest Common Divisor) (15p) HARD

Create a recursive function `gcd` that takes:

- two integers `a` and `b`

The function should find the Greatest Common Divisor of `a` and `b` using **Euclidean Algorithm** recursively.

Euclidean Algorithm:

```
gcd(a, b) = gcd(b, a % b)    (if b != 0)
gcd(a, 0) = a                (base case)
```

Example:

```
>> Enter first number: 48
>> Enter second number: 18
Calculating: gcd(48, 18) = gcd(18, 12)
Calculating: gcd(18, 12) = gcd(12, 6)
Calculating: gcd(12, 6) = gcd(6, 0)
Result: gcd(48, 18) = 6
```

BONUS Question – Print Binary (15p) HARD

Create a recursive function `print_binary` that takes:

- an integer `n`

The function should print the binary representation of `n` using recursion.

Hint:

- $n \% 2$ gives the last binary digit
- $n / 2$ removes the last binary digit
- Print in reverse order (use recursion wisely!)

Example:

```
>> Enter a number: 10
Binary: 1010
```

Another Example:

```
>> Enter a number: 25
Binary: 11001
```

Grading Distribution:

Difficulty	Questions	Points Each	Total
 Easy	Q1-Q6	10p	60p
 Medium	Q7-Q9	15p	45p
 Hard	Q10	20p	20p
 Bonus	GCD, Binary	15p each	30p (Extra)

Base Total: 125 points (25 bonus points available)

Hints

1. **Base Case:** Every recursive function must have a base case that stops the recursion.
2. **Recursive Case:** The function should call itself with a smaller/simpler version of the problem.

Example Recursive Structure:

```
int recursive_function(int n) {
    // Base case
    if (n == 0) return some_value;

    // Recursive case
    return n + recursive_function(n - 1);
}
```

Example with Printing Steps:

```
int factorial(int n) {  
    if (n <= 1) {  
        printf("Calculating: %d! = 1\n", n);  
        return 1;  
    }  
    printf("Calculating: %d! = %d × %d!\n", n, n, n-1);  
    return n * factorial(n - 1);  
}
```